

ENDOMETRIÔMETRO — Prompt para o Google AI Studio e Rotinas de Programação

Este documento tem duas partes: (1) o prompt pronto para colar no Google AI Studio e (2) as rotinas de programação (SQL no Supabase e componentes React) que sustentam o que o prompt pede. O prompt foi escrito para **não alterar layout nem paleta de cores** — só adicionar comportamento.

1. Prompt para o Google AI Studio

Você vai modificar o projeto ENDOMETRIÔMETRO (React + Supabase + Recharts) SEM alterar o layout visual nem a paleta de cores existente. Todas as mudanças abaixo são de COMPORTAMENTO e DADOS, mantendo os componentes visuais, espaçamentos, tipografia e cores atuais. Onde for preciso adicionar um elemento novo (ex.: campo de senha, checkbox de consentimento), reutilize os mesmos estilos de input/botão já usados nas telas existentes.

OBJETIVO GERAL

Hoje as telas "Pesquisa Individual" e "Área da Empresa / Cadastrar RH" são formulários de preenchimento direto, sem autenticação real. É preciso transformar isso em um fluxo de login/cadastro de verdade, corrigir a modelagem de dados e aplicar as regras de privacidade já definidas no projeto.

1. AUTENTICAÇÃO

- Adicionar campo de senha (mínimo 8 caracteres) nos dois formulários (individual e empresa), usando Supabase Auth (signUp / signInWithPassword).
- O formulário individual deixa de ter e-mail opcional: e-mail + senha passam a ser obrigatórios.
- O cadastro de empresa (hoje "Cadastrar Empresa e Gerar Link de Acesso") passa a exigir senha, para que a instituição consiga logar depois e acessar o próprio painel.
- O link "Entrar / Cadastrar" do cabeçalho deve levar para a MESMA tela de login usada pelos botões "Participar da pesquisa" e "Acessar área da instituição" – hoje são três caminhos descontraçados.
- Alternar entre "Entrar" e "Criar conta" no mesmo formulário (sem criar tela nova).

2. CONSENTIMENTO LGPD

- Substituir a caixa informativa de LGPD por um checkbox obrigatório: "Li e concordo com os Termos de Consentimento LGPD para uso dos meus dados de saúde nesta pesquisa."
- O botão "Próximo" do passo 1 do questionário fica desabilitado até o checkbox ser marcado.
- Aplicar o mesmo checkbox na seção "Vozes e Relatos" (depoimentos), com texto adaptado ao uso de publicação do relato.

3. VÍNCULO COM A EMPRESA

- Remover o campo de texto livre "Nome da Empresa em que trabalha" do formulário geral da pesquisa individual.
- O vínculo com empresa só deve existir quando a pessoa entra por um link institucional com token – nesse caso, manter o selo "Pesquisa Vinculada à Empresa: [nome]" que já existe.
- Quem responde pela home sem link de empresa fica com company_id nulo (não um texto digitado).

4. CAMPO DE LOCALIZAÇÃO

- Trocar o campo único "Cidade e Estado" (texto livre) por três campos: Cidade (texto), Bairro (texto) e Estado (lista de UFs).

5. INDICADORES E PRIVACIDADE

- Os indicadores da home pública (equivalentes aos itens 7, 8, 9 e 10 do projeto) só podem ser exibidos quando o total agregado de respostas atingir no mínimo 5 – abaixo disso, ocultar o número e manter o card visualmente igual, só sem o valor (placeholder neutro).
- Nenhum indicador pode mostrar número aberto por empresa individual na home pública – somente somas globais.
- Substituir os gráficos genéricos atuais da seção "Dados e Estatísticas" (prevalência genérica, eficácia de tratamentos etc.) pelos indicadores calculados a partir das respostas reais da plataforma, usando as rotinas de backend já implementadas (`get_global_indicators`, `get_survey_metrics_aggregated`).

6. CORREÇÃO VISUAL (sem mudar paleta/layout)

- Corrigir o overflow dos rótulos nos gráficos de pizza ("Prevalência" e "Distribuição por Faixa Etária"), que hoje aparecem cortados saindo do card. Ajustar apenas quebra de linha e tamanho de fonte da legenda, mantendo cores e posicionamento dos cards.

NÃO FAÇA:

- Não altere cores, fontes, espaçamento ou ordem das seções existentes.
- Não crie uma tela nova de login separada das abas atuais – reaproveite a estrutura de abas "Pesquisa Individual" / "Área da Empresa" já existente, apenas adicionando os campos e validações descritos acima.
- Não remova nenhum conteúdo educativo (seções "Entendendo a Endometriose" e "Vozes e Relatos").

2. Rotinas de Programação — SQL (Supabase / PostgreSQL)

2.1 Ajustes no schema

```
-- company_id nulo (não "zero") quando a pessoa não tem empresa vinculada
```

```
ALTER TABLE public.profiles  
  ALTER COLUMN company_id DROP NOT NULL;
```

```
ALTER TABLE public.profiles  
  ADD CONSTRAINT profiles_company_id_fkey  
  FOREIGN KEY (company_id) REFERENCES public.companies(id)  
  ON DELETE SET NULL;
```

```
-- survey_id obrigatório em toda resposta
```

```
ALTER TABLE public.survey_responses  
  ALTER COLUMN survey_id SET NOT NULL;
```

```
ALTER TABLE public.survey_responses  
  ADD CONSTRAINT survey_responses_survey_id_fkey  
  FOREIGN KEY (survey_id) REFERENCES public.surveys(id);
```

```
-- Uma resposta por pessoa, por pesquisa
```

```
ALTER TABLE public.survey_responses  
  ADD CONSTRAINT survey_responses_unique_per_survey  
  UNIQUE (user_id, survey_id);
```

2.2 RPC — métricas por empresa, com K-Anonymity

```
create or replace function public.get_survey_metrics_aggregated(  
  --
```

```

    p_survey_id uuid,
    p_company_id uuid
)
returns table(total_respostas int, total_com_sintoma int)
language plpgsql
security definer
as $$
declare
    v_total int;
begin
    select count(*) into v_total
    from public.survey_responses
    where survey_id = p_survey_id
        and company_id = p_company_id
        and status = 'concluida';

    if v_total < 5 then
        return query select null::int, null::int;
    else
        return query
            select
                v_total,
                count(*) filter (where (responses->>'suspeita_sintoma')::boolean = true)
            from public.survey_responses
            where survey_id = p_survey_id
                and company_id = p_company_id
                and status = 'concluida';
    end if;
end;
$$;

```

2.3 RPC — indicadores globais (home pública), com K-Anonymity

```

create or replace function public.get_global_indicators()
returns table(
    total_mulheres_pesquisadas int,
    total_com_sintoma int,
    total_empresas_pesquisadas int
)
language plpgsql
security definer
as $$
declare
    v_total int;
begin
    select count(*) into v_total
    from public.survey_responses
    where status = 'concluida';

    if v_total < 5 then
        return query select null::int, null::int, null::int;
    else
        return query
            select
                v_total,
                count(*) filter (where (responses->>'suspeita_sintoma')::boolean = true),
                count(distinct company_id)
            from public.survey_responses
            where status = 'concluida';
    end if;
end;
$$;

```

```

        from public.survey_responses
        where status = 'concluida';
    end if;
end;
$$;

```

2.4 View materializada (cache dos indicadores, sem travar leitura)

```

create materialized view public.mv_global_indicators as
select
    count(*) as total_mulheres_pesquisadas,
    count(*) filter (where (responses->>'suspeita_sintoma')::boolean = true) as total_com_sintoma,
    count(distinct company_id) as total_empresas_pesquisadas
from public.survey_responses
where status = 'concluida';

-- índice necessário para permitir REFRESH ... CONCURRENTLY (não bloqueia leitura)
create unique index mv_global_indicators_unique on public.mv_global_indicators ((1));

-- chamado por um cron job / Edge Function a cada N minutos:
refresh materialized view concurrently public.mv_global_indicators;

```

2.5 RLS — quem pode ler o quê

```

alter table public.survey_responses enable row level security;

create policy "Participante vê só a própria resposta"
on public.survey_responses for select
using (auth.uid() = user_id);

create policy "Empresa vê só respostas vinculadas a ela"
on public.survey_responses for select
using (
    company_id in (
        select company_id from public.profiles
        where id = auth.uid() and role = 'instituicao'
    )
);

```

3. Rotinas de Programação — Frontend (React)

3.1 Login compartilhado (individual / empresa), com alternância cadastro/entrar

```

// LoginForm.jsx
import { useState } from "react";
import { supabase } from "../lib/supabaseClient";

export default function LoginForm({ role }) { // role: 'participante' | 'instituicao'
    const [email, setEmail] = useState("");
    const [senha, setSenha] = useState("");
    const [modoCadastro, setModoCadastro] = useState(false);
    const [erro, setErro] = useState(null);

```

```

async function handleSubmit(e) {
  e.preventDefault();
  setErro(null);

  const auth = modoCadastro
    ? await supabase.auth.signUp({ email, password: senha })
    : await supabase.auth.signInWithPassword({ email, password: senha });

  if (auth.error) {
    setErro(auth.error.message);
    return;
  }

  if (modoCadastro && auth.data.user) {
    await supabase.from("profiles").insert({
      id: auth.data.user.id,
      email,
      role,
      company_id: null
    });
  }
}

return (
  <form onSubmit={handleSubmit} className="login-form">
    <input
      type="email" required value={email}
      onChange={(e) => setEmail(e.target.value)}
      placeholder="E-mail"
    />
    <input
      type="password" required minLength={8} value={senha}
      onChange={(e) => setSenha(e.target.value)}
      placeholder="Senha"
    />
    {erro && <p className="erro">{erro}</p>}
    <button type="submit">{modoCadastro ? "Cadastrar" : "Entrar"}</button>
    <button type="button" onClick={() => setModoCadastro(!modoCadastro)}>
      {modoCadastro ? "Já tenho conta" : "Criar conta"}
    </button>
  </form>
);
}

```

3.2 Consentimento LGPD obrigatório

```

function ConsentimentoLGPD({ aceito, onChange }) {
  return (
    <label className="lgpd-consent">
      <input
        type="checkbox" required checked={aceito}
        onChange={(e) => onChange(e.target.checked)}
      />
      Li e concordo com os Termos de Consentimento LGPD para uso dos meus dados
      de saúde nesta pesquisa.
    </label>
  );
}

```

```

}

// No botão "Próximo" do passo 1:
// <button disabled={!aceito}>Próximo</button>

```

3.3 Campo estruturado de localização (cidade / bairro / UF)

```

const UFS = ["AC", "AL", "AP", "AM", "BA", "CE", "DF", "ES", "GO", "MA", "MT", "MS", "MG",
  "PA", "PB", "PR", "PE", "PI", "RJ", "RN", "RS", "RO", "RR", "SC", "SP", "SE", "TO"];

function CampoLocalizacao({ cidade, bairro, uf, onChange }) {
  return (
    <>
      <label>Cidade *</label>
      <input required value={cidade} onChange={(e) => onChange({ cidade: e.target.value })} />

      <label>Bairro *</label>
      <input required value={bairro} onChange={(e) => onChange({ bairro: e.target.value })} />

      <label>Estado *</label>
      <select required value={uf} onChange={(e) => onChange({ uf: e.target.value })}>
        <option value="">Selecione</option>
        {UFS.map((u) => <option key={u} value={u}>{u}</option>)}
      </select>
    </>
  );
}

```

3.4 Correção do overflow dos gráficos (sem mudar paleta/layout)

```

/* Só contenção – nenhuma cor ou posição de card é alterada */
.stat-card { overflow: hidden; }

.stat-card .recharts-pie-label-text,
.stat-card .recharts-legend-item-text {
  font-size: 0.72rem;
  white-space: normal;
  word-break: break-word;
  max-width: 90%;
}

.stat-card .recharts-wrapper {
  max-width: 100%;
}

```

Observação: as rotinas acima assumem os nomes de tabela/coluna já usados na arquitetura documentada do projeto (profiles, companies, surveys, survey_responses, campo responses em JSONB com uma chave suspeita_sintoma). Ajuste os nomes caso o schema real do Supabase use nomenclatura diferente.

